

Dataset Evolution — Raw UART to Detector Verdict

SDV Fault-Prediction & Self-Healing · how one telemetry tick becomes an anomaly score · traced on real log data (leak injected at t=46083 ms)

This document follows the **same two ticks** through every transformation in the simulation pipeline (run_campaign.py → dataset.py → baseline.py): a **healthy tick** at t=45983 ms and the **first faulty tick** at t=46083 ms — the instant the memory leak armed. Every value shown is real, extracted from an actual Renode run.

STAGE 0 — Raw UART text (what the firmware prints)

Format: TELEM, time_ms, node, signal_id, seq, value. Signals: 1=heap_free, 2=heap_used, 5=loop_latency. Messy text glued with debug prints.

```
HEALTHY tick:
TELEM,45983,1,1,1368,8112          -> heap_free = 8112
TELEM,45983,1,2,1369,0             -> heap_used = 0
Delta work_ms = 0TELEM,45983,1,5,1370,0 -> loop_latency = 0 (signal 5)

FIRST FAULTY tick:
TELEM,46083,1,1,1371,7976          -> heap_free = 7976 (dropped!)
TELEM,46083,1,2,1372,136           -> heap_used = 136 (leak allocated!)
Delta work_ms = 0TELEM,46083,1,5,1373,0 -> loop_latency = 0
```

STAGE 1 — parse_rows(): text to clean tuples

Regex `^TELEM, (\d+), \dots` anchors at line start, so the 'Delta work_ms' prefix makes every signal-5 line fail to match -> loop_latency is dropped.

```
HEALTHY  TELEM,45983,1,1,...,8112  OK  -> (45983, 1, 8112)
          TELEM,45983,1,2,...,0    OK  -> (45983, 2, 0)
          Delta...TELEM,45983,1,5,0 NO MATCH -> DROPPED

FAULTY   TELEM,46083,1,1,...,7976  OK  -> (46083, 1, 7976)
          TELEM,46083,1,2,...,136  OK  -> (46083, 2, 136)
```

STAGE 2 — find_inject_ms(): locate the fault

Scans for the first heap_used > 0.

```
(45983, 2, 0)    -> 0, still healthy
(46083, 2, 136)  -> > 0 ! -> inject_ms = 46083
```

STAGE 3 — label(): write the labelled CSV (LONG format) [the generated .csv]

Each tuple gets a label; faulty heap_free rows also get true_ttf = value / 1280 (drain rate).

timestamp	signal	value	label	true_ttf	
45983	heap_free	8112	normal		(healthy)
45983	heap_used	0	normal		
46083	heap_free	7976	faulty	6.231	(7976/1280 = 6.231 s to death)
46083	heap_used	136	faulty		

STAGE 4 — load_episode(): tag the run

Adds an episode column so many runs can stack without mixing.

timestamp	signal	value	label	true_ttf	episode
45983	heap_free	8112	normal		0
46083	heap_free	7976	faulty	6.231	0

STAGE 5 — pivot_wide() step 1: snap timestamps to the 100 ms grid [the bug fix]

Rounds each timestamp to nearest 100 so a tick's signals share one grid point.

```
45983 -> /100 = 459.83 -> round 460 -> x100 = 46000
46083 -> /100 = 460.83 -> round 461 -> x100 = 46100
```

STAGE 6 — pivot_wide() step 2: LONG to WIDE

Each signal becomes a column; each tick becomes one row.

```
BEFORE (long, many rows/tick)      AFTER (wide, ONE row/tick)
timestamp signal  value              episode timestamp heap_free heap_used
46000     heap_free 8112    -->      0         46000      8112.0    0.0
46000     heap_used 0              0         46100      7976.0    136.0
46100     heap_free 7976
46100     heap_used 136
```

STAGE 7 — pivot_wide() step 3: collapse the faulty flag

A tick is faulty if ANY of its samples was faulty.

```
episode timestamp heap_free heap_used faulty
0         46000      8112.0    0.0      False (healthy)
0         46100      7976.0    136.0     True  (faulty)
```

STAGE 8 — add_features(): the NEW derived columns appear

Slopes = rate of change (smoothed over 10 ticks, per second). heap_free_slope is the fault's fingerprint; it ramps negative as the window fills.

```
timestamp heap_free heap_free_slope loop_latency loop_latency_slope faulty
45900     8112.0      0.0              0.0          0.0          False flat
46000     8112.0      0.0              0.0          0.0          False flat
46100     7976.0     -136.0            0.0          0.0          True  fires!
46200     7840.0     -272.0            0.0          0.0          True  steeper
46300     7704.0     -408.0            0.0          0.0          True  steeper
47000     6752.0    -1360.0            0.0          0.0          True  full drain rate
```

STAGE 9 — ts_split(): split by time, then scale to numbers

First 60% of healthy rows train; rest + all faulty test. Standardize with mu/sd from train-normal only. Healthy data is perfectly flat, so sd collapses to 1.

```
mu = [8112, 0, 0, 0]      sd = [1, 1, 1, 1]      z = (x - mu) / sd
```

```
HEALTHY [8112, 0, 0, 0] -> [ 0, 0, 0, 0]
FAULTY  [7976,-136, 0, 0] -> [-136, -136, 0, 0]
FAULTY  deep(47000)      -> [-1360, -1360, 0, 0]
```

```
shapes: Xtr_normal=(273,4) Xte=(237,4) faulty-in-test=54
```

STAGE 10 — the two detectors score every test row (real scores)

Both learned 'normal = [0,0,0,0]'. Isolation Forest has no spread to learn (sd all 1) and flatlines; the Autoencoder rebuilds healthy rows and the error explodes on faults.

```
ISOLATION FOREST      AUTOENCODER (reconstruction error)
healthy [0,0,0,0]     -> 0.5      healthy [0,0,0,0]     -> ~0.000
faulty  [-136,...]    -> 0.5      faulty  [-136,...]    -> ~8,867
faulty  [-1360,...]   -> 0.5      faulty  deep         -> 13,598,190
EVERYTHING = 0.5 (blind)      CLEAR separation
```

STAGE 11 — report(): the final grade

ROC-AUC = separability (0.5 random, 1.0 perfect). FP/hour = false alarms per hour of health.

```
alarm line (thr) = 99th pct of healthy error ~ 0.0000
```

```
[iso-forest ] ROC-AUC = 0.500   FP/hour = 0.00   (flat-data failure)
[denoise-AE ] ROC-AUC = 1.000   FP/hour = 0.00   (perfect, zero false alarms)
```

The whole evolution on one screen

Stage	Function	Shape / form	The faulty tracer becomes
0	firmware	raw UART text	TELEM, 46083, 1, 1, 1371, 7976
1	parse_rows	(t, sig, val) tuples	(46083, 1, 7976)
2	find_inject_ms	one number	inject_ms = 46083

Stage	Function	Shape / form	The faulty tracer becomes
3	label	CSV, long	46083, heap_free, 7976, faulty, 6.231
4	load_episode	long + episode	..., episode=0
5	pivot snap	grid time	46083 -> 46100
6	pivot	wide	46100 7976 136
7	label collapse	+ faulty flag	faulty=True
8	add_features	+ slopes	heap_free_slope = -136
9	ts_split	scaled arrays	[-136, -136, 0, 0]
10	IF / AE	scores	IF 0.5 · AE error 8867
11	report	grades	AE ROC 1.000

One sentence: raw text → clean tuples → labelled long CSV → wide one-row-per-tick → engineered slope features → time-split scaled numbers → two detectors, where the AE's reconstruction error separates leak from health perfectly and Isolation Forest goes blind because the healthy data is too flat.

Two findings from this trace: (1) loop_latency telemetry is silently dropped — the 'Delta work_ms' debug printf glues onto the signal-5 line and the regex rejects it; (2) the IF=0.5 / AE=1.0 result is 'clean-data' perfect and needs noise + multi-rate augmentation to be defensible. Generated for the SDV thesis (Maior Cristian-Alexandru, ULBS).